dfnWorks → GEOS Workbench

From zero on Windows — copy, paste, run

You do **not** need to know Git, Docker or Python. Follow the steps in order, paste one grey block at a time into PowerShell, press **Enter**, and wait for it to finish before the next block.

Total time: about **20 minutes** (most of it is one large download).

What you are installing

Program	Why	Where
Git	downloads (clones) the project code	https://git-scm.com/download/win
Python 3.10+	runs the panel	https://python.org/downloads
Docker Desktop	runs dfnWorks and GEOS without installing Linux tools	https://docker.com/products/docker-desktop

Plus two folders of code, downloaded in step 3.

> **What you do NOT need:** no GEOS source clone, no `thirdPartyLibs` folder, no C++ compiler, no Linux, no GPU. GEOS is only needed to *solve* the model (optional step 9); its libraries come from a container image. The `thirdPartyLibs` directory belongs to GEOS' own build system and is never used by this project.

Step 1 — Install the three programs

1. **Git** — run the installer, keep every default, click *Next* to the end.
2. **Python** — on the first installer screen **tick “Add python.exe to PATH”**, then *Install Now*.
3. **Docker Desktop** — install, accept the WSL2 prompt, restart the PC when asked. After restart, open **Docker Desktop** from the Start menu and wait until it says **Engine running** (whale icon in the tray, not animated).

> Docker Desktop must be running every time you use the panel.

Step 2 — Open PowerShell

Press **Win + X** → **Terminal** (or *Windows PowerShell*).

A window opens with a line ending in `>`. That is where you paste. To paste, right-click inside the window (or `Ctrl+V`), then press **Enter**.

Check the three programs answered correctly:

```
git --version
python --version
docker --version
```

Each must print a version number. If one says *"not recognized"*, that program did not install — see **Troubleshooting** at the end.

Step 3 — Download the project code

Two repositories are needed. Paste this whole block once:

```
mkdir C:\dfn-project
cd C:\dfn-project
git clone https://github.com/lanl/dfnWorks.git
git clone https://github.com/sadafiiman/dfn-geos-workbench.git
```

- `dfnWorks` — the fracture-network generator (used by the panel).
- `dfn-geos-workbench` — the panel, converters and report (this project).

> Use a short path like `C:\dfn-project`. Do **not** put it on the Desktop or in OneDrive: long paths and syncing break the mesh tools.

Step 4 — Install the Python package and the dfnWorks image

```
cd C:\dfn-project\dfn-geos-workbench
python -m pip install --upgrade pip
python -m pip install numpy
docker pull ees16/dfnworks:latest
```

The last line downloads about **1.5 GB**. Wait until it prints *"Status: Downloaded newer image"* or *"Image is up to date"*.

Step 5 — Start the panel

```
python run_panel.py
```

You should see:

```
dfn-GEOS Workbench panel → http://localhost:8765
```

If instead you see `cannot listen on port 8765`, another program owns that port — use `python run_panel.py --port 9000` and open http://localhost:9000. **Do not** keep browsing 8765 in that case: you would be looking at the other program, not the panel.

Leave this window **open** — it is the server. Now open your browser (Chrome/Edge/Firefox) and go to:

http://localhost:8765

To stop the panel later, click in the PowerShell window and press **Ctrl+C**.

Step 6 — Settings (do this once)

In the panel click **Settings** and enter exactly:

Field	Value
dfnWorks backend	`docker`
dfnWorks docker image	`ees16/dfnworks:latest`
dfnWorks repository	`C:\dfn-project\dfnWorks`
GEOS executable	leave **empty**

Click **Save**. The dashboard now shows the paths it will use.

Step 7 — Run your first case

1. Click **Quick process** (top bar).
2. Drag a **ZIP** onto the page — or click *pick files* and select your files.
3. Optionally type a case name.
4. Click **Process**.

The panel recognises what you gave it and runs the right steps:

Your input	What the panel does
`full_mesh.inp`, `params.txt`, `polys\`, `dfm_mesh\`	converts to a GEOS model
`dfn_driver.py`, `input_generator.dat`, `case_params.json`	generates the dfnWorks mesh, then converts
`*.xml` + `*.vtm` / `*.vtu`	runs GEOS on your own deck
nothing recognised	runs the whole pipeline with defaults

A normal case takes **2–10 minutes**. The page shows each stage turning green; the **Log** button next to a stage shows the full output.

The three stages, in one line each:

Stage	What it does	What it leaves behind
1 · dfnWorks mesh	builds the random fracture network and meshes it (dfnGen → LaGriT)	`output\full_mesh.inp`, `params.txt`, `polys\`, `dfm_mesh\`
2 · Convert → GEOS	translates those meshes into GEOS input	`geos_case\` (`.vtm`, `.vtu`, conformal deck) and `geos_case_edfm\geos_dfn_edfm.xml`
3 · Run GEOS	simulates flow in the rock + fractures (needs step 9)	`dfn_edfm.pvd` + one folder of `.vtu` per time step

> No input file yet? Use the example shipped with the project: open `C:\dfn-project\dfn-geos-workbench\poc\output`, select the files, and zip them — that is a real dfnWorks result you can convert immediately.

Step 8 — Collect the results

Open the case and click ⬇ **Download results (.zip)**, then unzip it.

```
geos_case\
  case.vtm                  conforming model: rock block + fracture surfaces
  main.vtu                  rock matrix (tetrahedra)
  fracture.vtu              explicit fracture surfaces (triangles + aperture)
  geos_dfn_fractures.xml    conformal GEOS input deck
geos_case_edfm\
  geos_dfn_edfm.xml         EDFM GEOS input deck — this is the one to run
dfn_edfm.pvd                the simulation time series (after step 9)
dfn_edfm\0000N0\...         one folder per time step (matrix + fractures)
output\                     dfnWorks files (generated or uploaded)
```

Looking at the results in ParaView

Install **ParaView** from https://paraview.org (free). Then:

To see	Open this file	Then
The simulation over time	`dfn_edfm.pvd`	**Apply**, press ▶ to animate t = 0 … 50 000 s
Pressure in the rock	(tree) `matrix/Level0/RockMatrix`	colour by **`pressure`**
The fracture network	(tree) `matrix/Level0/Fracture`	colour by `pressure` or `elementAperture`; use *Surface With Edges* to see the 2-D elements
Just the geometry	`geos_case\case.vtm`	colour by `fracture_id` (fractures) or `attribute` (rock)

> Open the **`.pvd`**, not the individual `.vtu`: one `.vtu` is a single time step of a single region.

If the colour dropdown looks empty, toggle the Point/Cell-array button next to it — GEOS writes cell data.

Everything here (and the full list of files, plus velocity and CSV output) is explained in **`docs\USER_GUIDE.md`, section 10**.

Step 9 — Optional: also run the GEOS solver

Everything above produces the **model**. Solving it needs a GEOS binary, which only exists for Linux. On Windows you have two choices:

- **WSL2 (easiest)** — install Ubuntu from the Microsoft Store, and follow `docs\INSTALL_ubuntu.md` inside it; run the panel from there too.
- **Docker** — build a GEOS image once and set *GEOS backend* = `docker`, *GEOS docker image* = `dfn-geos:1.1`, and leave *GEOS executable* empty (the panel uses `/opt/geos/bin/geosx` inside the image). Exact build commands: `docs\INSTALL_windows.md`, section 5.

Both options need the **GEOS source** (only to compile it once) but **not** `thirdPartyLibs`: the `geosx/ubuntu24.04-gcc13` image already contains every third-party library.

Without it, stage *3 · Run GEOS* fails fast with *"GEOS executable is not configured"* — that is expected and harmless: the mesh and decks are complete.

Step 10 — Next time you use it

Only two steps are needed after a restart:

```
cd C:\dfn-project\dfn-geos-workbench
python run_panel.py
```

…then open http://localhost:8765. Docker Desktop must be running.

Troubleshooting

What you see	What to do
`'git' is not recognized`	Reinstall Git, then **close and reopen** PowerShell
`'python' is not recognized`	Reinstall Python with *Add python.exe to PATH* ticked
`'docker' is not recognized`, or *cannot connect to the Docker daemon*	Start Docker Desktop and wait for *Engine running*
`python run_panel.py` → *No module named numpy*	Run `python -m pip install numpy` again
Port 8765 already in use	`python run_panel.py --port 9000`, then open port 9000
Browser shows gibberish / binary instead of the panel	You are on a port owned by another program — start with `--port 9000` and open that
Stage 1: *no intersecting fractures*	Make fractures larger, or lower *Mesh h*
Stage 1: `MO DOES NOT EXIST: mo_poisson_pts`	Old LaGriT found — keep *dfnWorks backend* = `docker`
Stage 3: *GEOS executable is not configured*	Expected without step 9 — and the image must be built first (`docker images` must list it). Without a solver you still get the mesh + decks
Case looks stuck at 0 %	Open **Log**; the first run pulls the image and can take minutes
Opening `dfn_edfm.pvd` shows **only XML text**	That is normal — it is data, not a picture. Open it in **ParaView** (step 8)
Files owned by `root` on Linux/WSL	`docker run --rm -v "$PWD":/w ees16/dfnworks:latest chown -R 1000:1000 /w`

One-page command summary

```
mkdir C:\dfn-project
cd C:\dfn-project
git clone https://github.com/lanl/dfnWorks.git
git clone https://github.com/sadafiiman/dfn-geos-workbench.git
cd C:\dfn-project\dfn-geos-workbench
python -m pip install numpy
docker pull ees16/dfnworks:latest
python run_panel.py
```

Settings: backend `docker` · image `ees16/dfnworks:latest` · repository `C:\dfn-project\dfnWorks`.

Full reference: `docs\USER_GUIDE.md` *(panel),* `docs\INSTALL_windows.md` *(platform + GEOS build),* `report\feasibility_report.md` *(technical results).*